

SLIP---1---1

```

import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        System.out.println("Multiplication Table for " + number +
            ":");
        for (int i = 1; i <= 10; i++) {
            int result = number * i;
            System.out.println(number + " x " + i + " = " + result);
        }

        scanner.close();
    }
}

```

// slip1-- ---2----

```

import java.util.LinkedList;
import java.util.Iterator;
import java.util.ListIterator;

public class ColorLinkedList {
    public static void main(String[] args) {
        // Step 1: Create a linked list with the given
        colors
        LinkedList<String> colorList = new
        LinkedList<>();
        colorList.add("Red");
        colorList.add("Blue");
        colorList.add("Yellow");
        colorList.add("Orange");

        // Step 2: Display contents using an Iterator
        System.out.println("Contents of the list using
        Iterator:");
        Iterator<String> iterator = colorList.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        // Step 3: Display contents in reverse using
        ListIterator
        System.out.println("\nContents of the list in
        reverse order using ListIterator:");
        ListIterator<String> listIterator =
        colorList.listIterator(colorList.size());
        while (listIterator.hasPrevious()) {
            System.out.println(listIterator.previous());
        }

        // Step 4: Create another list with "Pink" and
        "Green"
        LinkedList<String> newColors = new
        LinkedList<>();
        newColors.add("Pink");
        newColors.add("Green");

        // Step 5: Insert the new list elements
        between "Blue" and "Yellow"
        int insertIndex = colorList.indexOf("Yellow");
        colorList.addAll(insertIndex, newColors);

        // Step 6: Display the final contents of the list
        System.out.println("\nContents of the list after
        insertion:");
        for (String color : colorList) {
            System.out.println(color);
        }
    }
}

```

Slip—2—1

```
import java.util.*;

public class ArrayListExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of integers (n):
");
        int n = scanner.nextInt();

        ArrayList<Integer> arrayList = new ArrayList<>();

        System.out.println("Enter " + n + " integers:");
        for (int i = 0; i < n; i++) {
            int num = scanner.nextInt();
            arrayList.add(num);
        }

        System.out.println("Elements in the ArrayList:");
        for (int i = 0; i < arrayList.size(); i++) {
            System.out.println(arrayList.get(i));
        }

        scanner.close();
    }
}
```

SLIP—2-2

```
/ slip 2
// MyNumber.java
class MyNumber {
    // Private integer data member
    private int number;

    // Default constructor to initialize the number to 0
    public MyNumber() {
        this.number = 0;
    }

    // Constructor to initialize the number to a given
    value
    public MyNumber(int number) {
        this.number = number;
    }

    // Method to check if the number is negative
    public boolean isNegative() {
        return number < 0;
    }
}
```

```
// Method to check if the number is positive
public boolean isPositive() {
    return number > 0;
}

// Method to check if the number is odd
public boolean isOdd() {
    return number % 2 != 0;
}

// Method to check if the number is even
public boolean isEven() {
    return number % 2 == 0;
}

// Main method to use command-line arguments
public static void main(String[] args) {
    if (args.length != 1) {
        System.out.println("Please provide exactly one
integer as a command-line argument.");
        return;
    }

    try {
        // Parse the command-line argument to an
        integer
        int value = Integer.parseInt(args[0]);

        // Create an object of MyNumber using the
        parameterized constructor
        MyNumber myNum = new MyNumber(value);

        // Perform the required operations and display
        the results
        System.out.println("Number: " + value);
        System.out.println("Is Negative: " +
myNum.isNegative());
        System.out.println("Is Positive: " +
myNum.isPositive());
        System.out.println("Is Odd: " + myNum.isOdd());
        System.out.println("Is Even: " + myNum.isEven());

    } catch (NumberFormatException e) {
        System.out.println("Invalid input! Please provide
a valid integer.");
    }
}
```

SLIP---3---1

```

import java.util.Scanner;

public class SumOfArrayElements {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the value of n: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Please enter a valid positive integer for n.");
            return;
        }

        int[] numbers = new int[n]; // Create an array to store the numbers

        System.out.println("Enter " + n + " different numbers:");
        for (int i = 0; i < n; i++) {
            numbers[i] = sc.nextInt();
        }

        // Calculate the sum of the elements in the array
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += numbers[i];
        }

        System.out.println("Sum of the elements in the array is: " + sum);

        sc.close(); // Close the scanner to release resources
    }
}

```

Slip—3—2

```

// slip3
import java.util.Scanner;

class Account {
    int accno;
    String accname;
    double balance;

    // Constructor to initialize account details
    public Account(int accno, String accname, double balance) {
        this.accno = accno;
        this.accname = accname;
        this.balance = balance;
    }

    // Static method to sort accounts by balance
    public static void sortAccount(Account[] accounts) {

```

```

        for (int i = 0; i < accounts.length - 1; i++) {
            for (int j = 0; j < accounts.length - 1 - i; j++) {
                if (accounts[j].balance > accounts[j + 1].balance) {
                    // Swap accounts[j] and accounts[j + 1]
                    Account temp = accounts[j];
                    accounts[j] = accounts[j + 1];
                    accounts[j + 1] = temp;
                }
            }
        }
    }

    // Method to display account details
    public void display() {
        System.out.println("AccNo: " + accno + ", Name: " + accname + ", Balance: " + balance);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Get the number of accounts
        System.out.print("Enter the number of accounts: ");
        int n = scanner.nextInt();

        // Create an array of Account objects
        Account[] accounts = new Account[n];

        // Input account details
        for (int i = 0; i < n; i++) {
            System.out.println("Enter details for account " + (i + 1) + ":");
            System.out.print("Account No: ");
            int accno = scanner.nextInt();
            scanner.nextLine(); // Clear the buffer
            System.out.print("Account Name: ");
            String accname = scanner.nextLine();
            System.out.print("Balance: ");
            double balance = scanner.nextDouble();

            // Store the account in the array
            accounts[i] = new Account(accno, accname, balance);
        }

        // Sort the accounts by balance
        sortAccount(accounts);

        // Display sorted accounts
        System.out.println("\nAccounts sorted by balance:");
        for (Account account : accounts) {
            account.display();
        }

        scanner.close();
    }
}

```

Slip—4—1---

```
import java.util.Scanner;

public class GreetUser {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String usr = sc.nextLine();

        // Convert the user's name to uppercase
        String ucase = usr.toUpperCase();

        // Greet the user
        System.out.println("Hello " + ucase + ", nice to meet you!");

        sc.close();
    }
}
```

Slip---4—2

```
// slip4
import java.util.Scanner;

class Product {
    int id;
    String name;
    double price;

    // Constructor to initialize product details
    Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}
```

```
public class ProductArray {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Array to store 5 Product objects
        Product[] products = new Product[5];

        // Input product details from the user
        for (int i = 0; i < 5; i++) {
            System.out.println("Enter details for Product " + (i + 1) + ":");
            System.out.print("ID: ");
            int id = scanner.nextInt();
            scanner.nextLine(); // Consume newline
            System.out.print("Name: ");
            String name = scanner.nextLine();
            System.out.print("Price: ");
            double price = scanner.nextDouble();

            products[i] = new Product(id, name, price);
        }

        // Find the product with the minimum price
        Product minPriceProduct = products[0];
        for (int i = 1; i < products.length; i++) {
            if (products[i].price < minPriceProduct.price) {
                minPriceProduct = products[i];
            }
        }

        // Display the name of the product with the minimum price
        System.out.println("Product with the minimum price is: " + minPriceProduct.name);
    }
}
```

Slip---5—1

```

import java.util.Scanner;

class ZeroNumberException extends Exception {
    public ZeroNumberException() {
        super("Number is 0");
    }
}

public class FactorialCalculator {
    public static int calculateFactorial(int n) {
        if (n == 0) {
            return 1;
        } else {
            return n * calculateFactorial(n - 1);
        }
    }

    public static void main(String[] args) {
        try {
            Scanner scanner = new Scanner(System.in);
            System.out.print("Enter a number: ");
            int num = scanner.nextInt();

            if (num == 0) {
                throw new ZeroNumberException();
            } else {
                int result = calculateFactorial(num);
                System.out.println("Factorial of " + num + " is "
+ result);
            }
        } catch (ZeroNumberException e) {
            System.out.println(e.getMessage());
        } catch (java.util.InputMismatchException e) {
            System.out.println("Invalid input. Please enter a
valid number.");
        }
    }
}

```

Slip5—2—

```

// Base class: Point
class Point {
    int x, y;
    // Default constructor
    Point() {
        this.x = 0;
        this.y = 0;
    }
    // Parameterized constructor
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

```

// Method to display point details
void display() {
    System.out.println("Point Coordinates: (" + x + ", " +
y + ")");
}

// Subclass: ColorPoint
class ColorPoint extends Point {
    String color;

    // Parameterized constructor
    ColorPoint(int x, int y, String color) {
        super(x, y); // Call the parent class constructor
        this.color = color;
    }

    // Override display method to include color
    @Override
    void display() {
        super.display(); // Call the display method from
Point class
        System.out.println("Color: " + color);
    }
}

// Main class to test the functionality
public class Main {
    public static void main(String[] args) {
        // Create a Point object using default constructor
        Point p1 = new Point();
        p1.display();

        // Create a Point object using parameterized
constructor
        Point p2 = new Point(5, 10);
        p2.display();

        // Create a ColorPoint object
        ColorPoint cp = new ColorPoint(3, 4, "Red");
        cp.display();
    }
}

```

Slip—6—1

```

import java.util.*;

public class UniqueIntegerCollection {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        HashSet<Integer> integerSet = new HashSet<>();

        System.out.print("Enter the number of integers (n): ");
        int n = scanner.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("Enter an integer: ");
            int num = scanner.nextInt();
            integerSet.add(num);
        }

        // Convert the HashSet to a sorted list
        ArrayList<Integer> sortedList = new
ArrayList<>(integerSet);
        Collections.sort(sortedList);

        System.out.println("Unique integers in sorted order:");
        for (int num : sortedList) {
            System.out.print(num + " ");
        }
        System.out.println();

        System.out.print("Enter an integer to search for: ");
        int searchElement = scanner.nextInt();

        if (integerSet.contains(searchElement)) {
            System.out.println("The element " + searchElement +
"is in the collection.");
        } else {
            System.out.println("The element " + searchElement +
"is not in the collection.");
        }

        scanner.close();
    }
}

```

SLIP----6----2

```

// slip 6
import java.util.Scanner;

class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of employees: ");
        int n = scanner.nextInt();
        Employee[] employees = new Employee[n];

        // Input employee details
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Employee " +
(i + 1) + ":");
            System.out.print("ID: ");
            int id = scanner.nextInt();
            scanner.nextLine(); // Consume newline
            System.out.print("Name: ");
            String name = scanner.nextLine();
            System.out.print("Salary: ");
            double salary = scanner.nextDouble();

            employees[i] = new Employee(id, name, salary);
        }

        // Find employee with maximum salary
        Employee maxSalaryEmployee = employees[0];
        for (int i = 1; i < n; i++) {
            if (employees[i].salary > maxSalaryEmployee.salary)
            {
                maxSalaryEmployee = employees[i];
            }
        }

        System.out.println("\nEmployee with the highest
salary: " + maxSalaryEmployee.name);
        scanner.close();
    }
}

```

SLIP---7----1

```
import java.util.*;

public class EmployeeHashTable {
    public static void main(String[] args) {
        // Create a Hashtable to store employee details
        Hashtable<String, Double> employeeSalaries = new
        Hashtable<>();

        // Add employee names and salaries to the Hashtable
        employeeSalaries.put("Ravi", 50000.0);
        employeeSalaries.put("Kiran", 60000.0);
        employeeSalaries.put("Vrushabh", 55000.0);
        employeeSalaries.put("Jatin", 70000.0);

        // Display the details of the Hashtable
        System.out.println("Employee Details:");

        // Using Enumeration to iterate through keys and values
        Enumeration<String> keys = employeeSalaries.keys();
        while (keys.hasMoreElements()) {
            String name = keys.nextElement();
            double salary = employeeSalaries.get(name);
            System.out.println("Name: " + name + ", Salary: " +
salary);
        }
    }
}
```

SLIP----7-----2

```
import java.util.Scanner;

class Student {
    int rollNo;
    String name;
    double percentage;

    // Default constructor
    Student() {
        this.rollNo = 0;
        this.name = "Unknown";
        this.percentage = 0.0;
    }
}
```

```
// Parameterized constructor
Student(int rollNo, String name, double percentage) {
    this.rollNo = rollNo;
    this.name = name;
    this.percentage = percentage;
}

// Method to display student details
void display() {
    System.out.println("Roll No: " + rollNo + ", Name: " +
name + ", Percentage: " + percentage + "%");
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Student[] students = new Student[5];

        // Input details for 5 students using a normal for loop
        for (int i = 0; i < 5; i++) {
            System.out.println("\nEnter details for Student " + (i +
1) + " :");
            System.out.print("Roll No: ");
            int rollNo = scanner.nextInt();
            scanner.nextLine(); // Consume newline
            System.out.print("Name: ");
            String name = scanner.nextLine();
            System.out.print("Percentage: ");
            double percentage = scanner.nextDouble();

            // Create and store the student object
            students[i] = new Student(rollNo, name, percentage);
        }
        // Display details of all students using a normal for loop
        System.out.println("\nStudent Details:");
        for (int i = 0; i < 5; i++) {
            students[i].display();
        }
        scanner.close();
    }
}
```

Slip—8---1

```

public class ReverseNumber {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Usage: java ReverseNumber
<number>");
            return;
        }
        // Parse the number from the command line argument
        int number = Integer.parseInt(args[0]);

        // Reverse the number
        int reversedNumber = reverseNumber(number);

        System.out.println("Original number: " + number);
        System.out.println("Reversed number: " +
reversedNumber);
    }
    public static int reverseNumber(int number) {
        int reversedNumber = 0;
        while (number != 0) {
            int digit = number % 10;
            reversedNumber = reversedNumber * 10 + digit;
            number = number / 10;
        }
        return reversedNumber;
    }
}

```

SLIP—8—2---

```

// slip 8
import java.util.Scanner;

// Custom exception class to handle invalid dates
class InvalidDateException extends Exception {
    public InvalidDateException(String message) {
        super(message);
    }
}

// MyDate class with day, month, and year fields
class MyDate {
    private int day;
    private int month;
    private int year;

    public MyDate() {
        this.day = 1;
        this.month = 1;
        this.year = 2000;
    }

    public void acceptDate() throws InvalidDateException {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the day (dd): ");

```

```

        int inputDay = scanner.nextInt();
        System.out.print("Enter the month (mm): ");
        int inputMonth = scanner.nextInt();
        System.out.print("Enter the year (yyyy): ");
        int inputYear = scanner.nextInt();
        if (isValidDate(inputDay, inputMonth, inputYear)) {
            this.day = inputDay;
            this.month = inputMonth;
            this.year = inputYear;
        } else {
            throw new InvalidDateException("Invalid date
input.");
        }
    }

    private boolean isValidDate(int day, int month, int year) {
        if (year < 1 || month < 1 || month > 12 || day < 1 || day
> getDaysInMonth(month, year)) {
            return false;
        }
        return true;
    }

    private int getDaysInMonth(int month, int year) {
        int[] daysInMonth = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30,
31, 30, 31};

        if (month == 2 && isLeapYear(year)) {
            return 29;
        } else {
            return daysInMonth[month];
        }
    }

    private boolean isLeapYear(int year) {
        if (year % 4 == 0) {
            if (year % 100 == 0) {
                return year % 400 == 0;
            } else {
                return true;
            }
        }
        return false;
    }

    public void displayDate() {
        System.out.println("Date: " + day + "/" + month + "/" +
year);
    }
}

public class DateExample {
    public static void main(String[] args) {
        MyDate myDate = new MyDate();
        try {
            myDate.acceptDate();
            myDate.displayDate();
        } catch (InvalidDateException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```


SLIP---9---1

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class PerfectNumCheck {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new
        InputStreamReader(System.in));

        try {
            System.out.print("Enter a number: ");
            int number = Integer.parseInt(reader.readLine());

            if (isPerfectNumber(number)) {
                System.out.println(number + " is a perfect number.");
            } else {
                System.out.println(number + " is not a perfect
number.");
            }
        } catch (NumberFormatException | IOException e) {
            System.out.println("Invalid input. Please enter a valid
positive integer.");
        }
    }

    public static boolean isPerfectNumber(int number) {
        if (number <= 0) {
            return false;
        }
        int sum = 0;
        for (int i = 1; i <= number / 2; i++) {
            if (number % i == 0) {
                sum += i;
            }
        }
        return sum == number;
    }
}

```

SLIP---9---2

```

class Point {
    protected double x;
    protected double y;

    public Point() {
        this.x = 0.0;
        this.y = 0.0;
    }
}

```

```

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public void display() {
        System.out.println("Point Coordinates: (" + x + ", " + y +
        ")");
    }
}

class Point3D extends Point {
    private double z;

    public Point3D() {
        super(); // Calls the default constructor of the superclass
        this.z = 0.0;
    }

    public Point3D(double x, double y, double z) {
        super(x, y); // Calls the parameterized constructor of the
superclass
        this.z = z;
    }

    @Override
    public void display() {
        System.out.println("3D Point Coordinates: (" + x + ", " + y
        + ", " + z + ")");
    }
}

public class PointDemo {
    public static void main(String[] args) {
        Point point2D = new Point(2.0, 3.0);
        point2D.display();

        Point3D point3D = new Point3D(1.0, 2.0, 3.0);
        point3D.display();
    }
}

```

SLIP—10—1

```
import java.util.Scanner;

public class PrimeNumberChecker {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Prompt the user to enter a number
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();
        // Close the scanner
        scanner.close();
        // Check if the number is prime
        boolean isPrime = isPrimeNumber(number);
        // Display the result
        if (isPrime) {
            System.out.println(number + " is a prime number.");
        } else {
            System.out.println(number + " is not a prime
number.");
        }
    }

    // Function to check if a number is prime
    public static boolean isPrimeNumber(int num) {
        if (num <= 1) {
            return false;
        }
        if (num <= 3) {
            return true;
        }

        // Check for divisibility by numbers from 2 to the square
        root of the number
        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                return false;
            }
        }

        return true;
    }
}
```

SLIP—10—2—

```
import java.util.Scanner;
// User-defined exception for insufficient funds
class InsufficientFundException extends Exception {
    public InsufficientFundException(String message) {
        super(message);
    }
}
class SavingAccount {
    int acno;
    String name;
    double balance;
```

```
// Constructor to initialize account details
SavingAccount(int acno, String name, double balance) {
    this.acno = acno;
    this.name = name;
    this.balance = balance;
}
// Deposit method
void deposit(double amount) {
    balance += amount;
    System.out.println("Deposited: " + amount);
}
// Withdraw method with exception handling
void withdraw(double amount) throws
InsufficientFundException {
    if (balance - amount < 500) {
        throw new InsufficientFundException("Minimum
balance of 500 must be maintained!");
    }
    balance -= amount;
    System.out.println("Withdrawn: " + amount);
}
// View balance method
void viewBalance() {
    System.out.println("Current Balance: " + balance);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Creating a sample account
        SavingAccount account = new SavingAccount(101, "Alice",
1000);
        System.out.println("Choose an operation: 1-Deposit, 2-
Withdraw, 3-View Balance");
        System.out.print("Enter choice: ");
        int choice = scanner.nextInt();
        try {
            switch (choice) {
                case 1:
                    System.out.print("Enter amount to deposit: ");
                    double depositAmount = scanner.nextDouble();
                    account.deposit(depositAmount);
                    break;
                case 2:
                    System.out.print("Enter amount to withdraw: ");
                    double withdrawAmount = scanner.nextDouble();
                    account.withdraw(withdrawAmount);
                    break;
                case 3:
                    account.viewBalance();
                    break;
                default:
                    System.out.println("Invalid choice!");
            }
        } catch (InsufficientFundException e) {
            System.out.println("Error: " + e.getMessage());
        }
        scanner.close();
    }
}
```

SLIP---11—1—

```
class MyDate {
    private int dd;
    private int mm;
    private int yy;

    // Parameterized constructor
    public MyDate(int dd, int mm, int yy) {
        this.dd = dd;
        this.mm = mm;
        this.yy = yy;
    }

    // Display the date in dd-mm-yy format
    public void displayDate() {
        System.out.println(this.dd + "/" + this.mm + "/" + this.yy);
    }

    public static void main(String[] args) {
        MyDate date = new MyDate(4, 11, 2023);
        System.out.print("Date: ");
        date.displayDate();
    }
}
```

SLIP---11—2—

```
abstract class Shape {
    public abstract double area();
    public abstract double volume();
}

class Sphere extends Shape {
    private double radius;

    public Sphere(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return 4 * Math.PI * Math.pow(radius, 2);
    }

    @Override
    public double volume() {
        return (4.0 / 3.0) * Math.PI * Math.pow(radius, 3);
    }

    public void display() {
        System.out.println("Sphere with radius " + radius);
        System.out.println("Area: " + area());
        System.out.println("Volume: " + volume());
    }

    public static void main(String[] args) {
        Sphere sp = new Sphere(3.5);
        sp.display();
    }
}
```

SLIP—12—1

```
// Series/SquareSeries.java
```

```
package Series;
```

```
public class SquareSeries {
    public static void printSquareSeries(int n) {
        for (int i = 1; i <= n; i++) {
            int square = i * i;
            System.out.print(square + " ");
        }
        System.out.println(); // Print a new line after the series
    }
}
```

<Second file>

```
// Main program
```

```
import Series.SquareSeries;
```

```
public class SqrSeriesEx {
    public static void main(String[] args) {
        int n = 10; // Change the value of n as needed
        System.out.println("Square Series of first " + n + "
terms:");
        SquareSeries.printSquareSeries(n);
    }
}
```

SLIP—12—2—

```
// Define the abstract class Shape
```

```
abstract class Shape {
    public abstract double area();
    public abstract double volume();
}
```

```
// Define the Cylinder class that extends Shape
```

```
class Cylinder extends Shape {
    private double radius;
    private double height;
```

```
    public Cylinder(double radius, double height) {
        this.radius = radius;
        this.height = height;
    }
```

```
// Implement the area method for Cylinder
```

```
@Override
```

```
public double area() {
    // Calculate the surface area of the cylinder
    double baseArea = Math.PI * radius * radius;
    double lateralArea = 2 * Math.PI * radius * height;
    return 2 * baseArea + lateralArea;
}
```

```
// Implement the volume method for Cylinder
```

```
@Override
```

```
public double volume() {
    // Calculate the volume of the cylinder
    return Math.PI * radius * radius * height;
}
```

```
    public static void main(String[] args) {
        // Create a Cylinder object with a specific radius and
height
        Cylinder cy = new Cylinder(2.4, 3.8);

        // Calculate and display the area and volume
        System.out.println("Cylinder Area: " + cy.area());
        System.out.println("Cylinder Volume: " + cy.volume());
    }
}
```

SLIP—13---1

```
import java.util.LinkedList;
import java.util.Iterator;

public class FruitList {
    public static void main(String[] args) {
        // Step 1: Create a linked list
        LinkedList<String> fruitList = new LinkedList<>();

        // Step 2: Add the names of fruits to the linked list
        fruitList.add("Apple");
        fruitList.add("Banana");
        fruitList.add("Guava");
        fruitList.add("Orange");

        // Step 3: Use an iterator to display the contents of the list
        Iterator<String> iterator = fruitList.iterator();
        System.out.println("Fruits in the list:");
        while (iterator.hasNext()) {
            String fruit = iterator.next();
            System.out.println(fruit);
        }
    }
}
```

SLIP—13—2

```
interface Operation {
    double PI = 3.142; // Constant PI

    double area(); // Method to calculate area
    double volume(); // Method to calculate volume
}

class Circle implements Operation {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return PI * radius * radius; // Area of a circle:  $\pi * r^2$ 
    }

    @Override
    public double volume() {
        // Circles are 2D, so volume is not applicable; return 0.
        return 0;
    }

    public void display() {
        System.out.println("Circle with radius " + radius);
        System.out.println("Area: " + area());
        System.out.println("Volume: " + volume());
    }
}

public class MainCircle {
    public static void main(String[] args) {
        double circleRadius = 2.5;
        Circle circle = new Circle(circleRadius);
        circle.display();
    }
}
```

SLIP---14—1

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JDBCConn {
    public static void main(String[] args) {
        String jdbcURL =
"jdbc:postgresql://localhost:5432/javadb";
        String username = "postgres";
        String password = "491625";

        try {
            // Load the PostgreSQL JDBC driver
            Class.forName("org.postgresql.Driver");

            // Create a connection to the database
            Connection connection =
DriverManager.getConnection(jdbcURL, username, password);

            if (connection != null) {
                System.out.println("Connected to the PostgreSQL
database successfully!");
                // You can perform database operations here

                // Don't forget to close the connection when done
                connection.close();
            }
        } catch (ClassNotFoundException e) {
            System.err.println("PostgreSQL JDBC driver not found.
Make sure it's in the classpath.");
            e.printStackTrace();
        }
        catch (SQLException e) {
            System.err.println("Connection to the database
failed.");
            e.printStackTrace();
        }
    } }

/* -- sudo apt install libpostgresql-jdbc-java
-- export CLASSPATH=$CLASSPATH:<path to the
postgresql.jar file>
-- sudo nano /etc/postgresql/<version>/main/pg_hba.conf
scroll down and here make following changes
-- Change IPV4 Connection Method from 'peer' or 'md5' to
'trust'
-- Change 'Database Administrative Login by Unix Domain
socket' method from 'peer' or 'md5' to 'trust'
-- sudo systemctl restart postgresql */

```

SLIP---14—2---

```

// Define the Operation interface
interface Operation {
    float PI = 3.142f; // Constant PI as a float

    float area();

    float volume();
}

// Implement the Operation interface in the Cylinder class
class Cylinder implements Operation {
    private float radius;
    private float height;

    // Constructor to initialize the radius and height of the
cylinder
    public Cylinder(float radius, float height) {
        this.radius = radius;
        this.height = height;
    }

    // Implement the area method to calculate the surface
area of the cylinder
    @Override
    public float area() {
        return 2 * PI * radius * (radius + height);
    }

    // Implement the volume method to calculate the volume
of the cylinder
    @Override
    public float volume() {
        return PI * radius * radius * height;
    }

    // Display the area and volume of the cylinder
    public void displayAreaAndVolume() {
        System.out.println("Cylinder Area: " + area());
        System.out.println("Cylinder Volume: " + volume());
    }

    public static void main(String[] args) {
        float cylinderRadius = 5.0f; // Set the radius as a float
        float cylinderHeight = 10.0f; // Set the height as a float

        // Create a Cylinder object
        Cylinder cylinder = new Cylinder(cylinderRadius,
cylinderHeight);

        // Display the area and volume of the cylinder
        cylinder.displayAreaAndVolume();
    }
}

```

SLIP—15—1

```
import java.util.LinkedList;
import java.util.List;
import java.util.ListIterator;

public class FruitLinkedList {
    public static void main(String[] args) {
        // Create a LinkedList of Fruits
        List<String> fList = new LinkedList<>();
        fList.add("Apple");
        fList.add("Banana");
        fList.add("Guava");
        fList.add("Orange");

        // Display the contents of the List in reverse order
        ListIterator<String> lit = fList.listIterator(fList.size());

        System.out.println("Fruits in reverse order:");
        while (lit.hasPrevious()) {
            System.out.println(lit.previous());
        }
    }
}
```

SLIP—15—2

```
class Employee {
    String name;
    double salary;
    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

class Developer extends Employee {
    String proj;

    public Developer(String name, double salary, String proj) {
        super(name, salary);
        this.proj = proj;
    }

    public void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Salary: Rs. " + salary);
        System.out.println("Project Name: " + proj);
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Developer developer = new Developer("Madan Gaikwad",
        60000, "Project X");
        developer.displayDetails();
    }
}
```

SLIP---16—1

```

public class MyNumber {
    private int value;

    public MyNumber(int value) {
        this.value = value;
    }

    public boolean isEven() {
        return value % 2 == 0;
    }

    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Please provide one integer
as a command line argument.");
            return;
        }

        try {
            int inputValue = Integer.parseInt(args[0]);
            MyNumber myNumber = new
MyNumber(inputValue);
            System.out.println("The Number " + inputValue +
" is even ? " + myNumber.isEven());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please provide
a valid integer as a command line argument.");
        }
    }
}

```

SLIP---17—2**<Employee.java>**

```

public class Employee {
    protected String name;
    protected double salary;

    // Constructor to initialize Employee details
    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    // Method to display Employee details
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }
}

```

2nd file**<Programmer.java>**

```

public class Programmer extends Employee {
    private String proglanguage;

    // Constructor to initialize Programmer details
    public Programmer(String name, double salary, String
proglanguage) {
        super(name, salary); // Call to the superclass
        constructor
        this.proglanguage = proglanguage;
    }

    // Method to display Programmer details
    @Override
    public void displayDetails() {
        super.displayDetails(); // Display Employee details
        System.out.println("Programming Language: " +
proglanguage);
    }
}

```

3rd java**<Main.java>**

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input for Employee and Programmer details
        System.out.print("Enter name: ");
        String name = scanner.nextLine();

        System.out.print("Enter salary: ");
        double salary = scanner.nextDouble();
        scanner.nextLine(); // Consume the leftover
        newline

        System.out.print("Enter programming language: ");
        String proglanguage = scanner.nextLine();

        // Create an object of Programmer class
        Programmer programmer = new
Programmer(name, salary, proglanguage);

        // Display the details of the Programmer
        System.out.println("\nProgrammer Details:");
        programmer.displayDetails();
    }
}

```

-----RUN COMMAND-----

```

javac Employee.java Programmer.java Main.java
java Main

```


SLIP---17----1---

```
public class MyNumber {
    private int number;

    public MyNumber(int value) {
        this.number = value;
    }

    public boolean isOdd() {
        return number % 2 != 0;
    }

    public static void main(String[] args) {
        if (args.length == 1) {
            try {
                int value = Integer.parseInt(args[0]);
                MyNumber myNumber = new MyNumber(value);

                if (myNumber.isOdd()) {
                    System.out.println(value + " is an odd number.");
                } else {
                    System.out.println(value + " is an even number.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Invalid input. Please provide a
valid integer.");
            }
        } else {
            System.out.println("Please enter any number at
command line.");
        }
    }
}
```

SLIP—17—2---

```
public class Student {
    private int rollno;
    private String name;
    private static int objectCount = 0;

    public Student() {
        objectCount++;
    }

    public Student(int rollno, String name) {
        this.rollno = rollno;
        this.name = name;
        objectCount++;
        System.out.println("Student object created. Object count:
" + objectCount);
    }

    public static void main(String[] args) {
        Student student1 = new Student(101, "Manish");
        Student student2 = new Student(102, "Nitin");

        System.out.println("Total Student objects created: " +
objectCount);
    }
}
```

SLIP---18---1

```
public class FactorsOfNumber {
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Please provide a single
number as a command-line argument.");
            return;
        }

        try {
            int number = Integer.parseInt(args[0]);
            if (number < 1) {
                System.out.println("Please enter a positive
integer.");
                return;
            }

            System.out.print("Factors of " + number + ": ");
            for (int i = 1; i <= number; i++) {
                if (number % i == 0) {
                    System.out.print(i + " ");
                }
            }
            System.out.println();
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a
valid integer.");
        }
    }
}
```

SLIP---18---2

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class UpperCase {
    public static void main(String[] args) {
        String filePath = "abc.txt";

        try (BufferedReader br = new BufferedReader(new
FileReader("abc.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                // Convert each line to uppercase and print
                System.out.println(line.toUpperCase());
            }
        } catch (IOException e) {
            System.err.println("An error occurred while
reading the file: " + e.getMessage());
        }
    }
}
```

SLIP—19---1

```

import java.util.Scanner;

public class MaxNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Ask the user to enter the size of array
        System.out.print("Enter the size of array: ");
        int n = sc.nextInt();

        // Create an array of size 'n'
        int[] numbers = new int[n];

        // Prompt the user to enter 'n' different numbers
        for (int i = 0; i < n; i++) {
            System.out.print("Enter number " + (i + 1) + ": ");
            numbers[i] = sc.nextInt();
        }

        // Find the maximum number in the array
        int max = numbers[0];
        for (int i = 1; i < n; i++) {
            if (numbers[i] > max) {
                max = numbers[i];
            }
        }

        // Display the maximum number
        System.out.println("The maximum number in the
array is: " + max);

        // Close the scanner
        sc.close();
    }
}

```

SLIP---19—2---

```

import java.util.Scanner;

abstract class Order {
    protected int id;
    protected String description;

    public abstract void accept();
    public abstract void display();
}

class PurchaseOrder extends Order {
    private String customerName;

    public void accept() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter Order ID: ");
        id = scanner.nextInt();
        scanner.nextLine(); // Consume the newline
character
        System.out.print("Enter Order Description: ");
        description = scanner.nextLine();
        System.out.print("Enter Customer Name: ");
        customerName = scanner.nextLine();
    }

    public void display() {
        System.out.println("Order ID: " + id);
        System.out.println("Order Description: " +
description);
        System.out.println("Customer Name: " +
customerName);
    }
}

public class OrdersInfo {
    public static void main(String[] args) {
        PurchaseOrder[] orders = new PurchaseOrder[3];

        for (int i = 0; i < 3; i++) {
            System.out.println("Enter details for Purchase
Order " + (i + 1));
            orders[i] = new PurchaseOrder();
            orders[i].accept();
        }

        System.out.println("\nDisplaying Purchase
Orders:");
        for (PurchaseOrder order : orders) {
            order.display();
            System.out.println();
        }
    }
}

```

SLIP---20---1

```

import java.util.Arrays;

public class SortNum {
    public static void main(String[] args) {
        if (args.length != 3) {
            System.out.println("Please provide exactly 3
numbers as command-line arguments.");
            return;
        }

        // Convert the command-line arguments to
integers
        int[] numbers = new int[3];
        for (int i = 0; i < 3; i++) {
            try {
                numbers[i] = Integer.parseInt(args[i]);
            } catch (NumberFormatException e) {
                System.out.println("Invalid input. Please
provide valid numbers as arguments.");
                return;
            }
        }

        // Sort the numbers
        Arrays.sort(numbers);

        // Display the sorted numbers
        System.out.print("Sorted Numbers: ");
        for (int number : numbers) {
            System.out.print(number + " ");
        }
        System.out.println();
    }
}

```

SLIP---20---2

```

public class Employee {
    private int id;
    private String name;
    private String deptName;
    private double salary;
    private static int objectCount = 0;

    public Employee() {
        this.id = 0;
        this.name = " ";
        this.deptName = " ";
        this.salary = 0.0;
        objectCount++;
    }

    public Employee(int id, String name, String
deptName, double salary) {
        this.id = id;
        this.name = name;
        this.deptName = deptName;
        this.salary = salary;
        objectCount++;
    }

    public static int getObjectCount() {
        return objectCount;
    }

    public void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Employee Name: " + name);
        System.out.println("Department Name: " +
deptName);
        System.out.println("Salary: " + salary);
    }

    public static void main(String[] args) {
        Employee emp1 = new Employee(1, "Vishal Dhare",
"Sales", 50000.0);
        System.out.println("Object Count: " +
Employee.getObjectCount());
        emp1.display();

        Employee emp2 = new Employee(2, "Vijay
Kulkarni", "RND", 60000.0);
        System.out.println("Object Count: " +
Employee.getObjectCount());
        emp2.display();
    }
}

```